

สรุปเทคโนโลยีที่ใช้ (Tech Stack)

เว็บแอป Photo Booth อิมเมจเมกนตร์ ทำงานในเบราว์เซอร์ทั้งหมด ไม่มี backend ไม่มีฐานข้อมูล และไม่ต้องใช้อินเทอร์เน็ตตอนใช้งาน

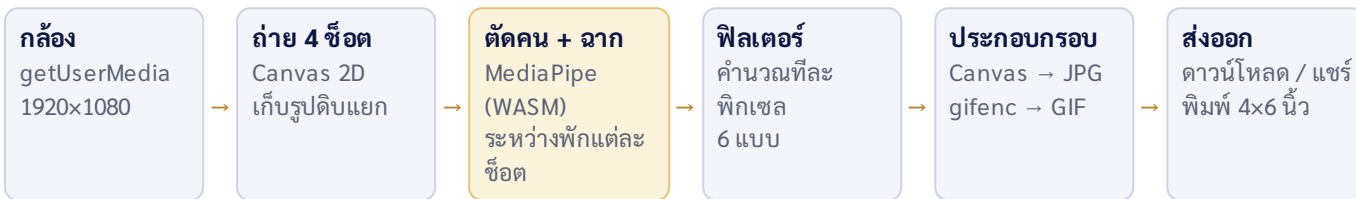
Repo: gorawit-
ui/photobooth

Branch: claude/magical-photo-booth-
eefh90

อัปเดต: 30 ก.ย. 2026 (commit
34fa628)

1 ภาพรวมสถาปัตยกรรม

ทุกขั้นตอนตั้งแต่เปิดกล้องจนได้ไฟล์ทำในเบราว์เซอร์ของเครื่องที่ติดตั้งได้แก่ คอม (Chrome/Edge/Safari) หรือ iPad (Safari) รูปจึงไม่ถูกส่งออกไปที่ไหน



2 แพ็กเกจหลัก (Core Stack)

ส่วน	เทคโนโลยี	เวอร์ชัน	ใช้ทำอะไร
ภาษา	TypeScript	7.0.2	ภาษาหลักของโค้ดทั้งหมด (strict mode)
UI	React + React DOM	19.3.0	หน้าจอ 5 หน้า: หน้าแรก > เลือกกรอบ > ถ่าย > ผลลัพธ์ > ดาวน์โหลด
Build / Dev server	Vite	8.3.1	<code>npm run dev</code> สำหรับพัฒนา, <code>npm run build</code> สร้างไฟล์ไปที่ <code>dist/</code>
Vite plugins	@vitejs/plugin-react	6.1.1	รองรับ React / JSX
	@vitejs/plugin-basic-ssl	2.3.0	เปิด HTTPS (self-signed) เพื่อให้ iPad ใช้กล้องผ่าน Wi-Fi ได้
	mediapipe-wasm เขียนเอง	—	ส่งไฟล์ WASM ของ MediaPipe ตอน dev และคัดลอกไปที่ <code>dist/</code> ตอน build (ไม่เก็บใน git)
สร้าง GIF	gifenc	1.0.3	เข้ารหัส GIF บนเครื่อง 600 px กว้าง, 4 เฟรม, 600 ms, เล่นวน
AI ตัดคน	@mediapipe/tasks-vision (Google)	1.0.1	Image Segmenter ทำงานบน WASM/CPU โหลดเมื่อใช้งาน (lazy)
โมเดล AI	selfie_multiclass_256x256.tflite	16 MB	แยก ผม / ผิว / เสื้อผ้า / พื้นหลัง เก็บใน <code>public/models/</code> ใช้ได้แม้ไม่มีเน็ต
Runtime (ตอนพัฒนา)	Node.js	≥ 20.19	ใช้ตอนพัฒนาและ build เท่านั้น แนะนำ 22 LTS (ตอนใช้งานไม่ต้องใช้)
Type definitions	@types/react, @types/node	19 / 26	ใช้ตรวจ type ตอนพัฒนา

3 ความสามารถของเบราว์เซอร์ที่ใช้ (Web APIs)

API	ใช้ทำอะไร
getUserMedia / enumerateDevices	เปิดกล้อง (ค่าเริ่มต้นกล้องหน้า) และเมนูเลือกกล้องเมื่อมีหลายตัว
Canvas 2D	ถ่ายภาพ, ครอปแบบ cover, วางคนบนฉาก, ฟิลเตอร์สี, ประกอบกรอบ, ประกายบนพริ้ว
WebAssembly	รันโมเดล AI ตัดคน (MediaPipe + XNNPACK บน CPU)
Web Audio	สร้างเสียงนักร้องหลัง / ซัดเตอร์ / กริ่ง ด้วยโค้ด ไม่ใช่ไฟล์เสียง
Fullscreen API	ปุ่มเต็มจอบนคอม (มี fallback สำหรับ Safari รุ่นเก่า)
Screen Wake Lock	กันจอตบระหว่างใช้งาน ถ้าเบราว์เซอร์ไม่รองรับจะข้ามไป
Web Share API	ปุ่ม "บันทึกรูปภาพ / แชร์" บน iPad
CSS @page + window.print()	พิมพ์รูปรวมขนาด 4x6 นิ้วแนวนอน (กดพิมพ์เองเท่านั้น)
localStorage	จำค่าเปิด/ปิดเสียง
Web App Manifest	เพิ่มลงหน้าโฮม iPad ให้เปิดเต็มจอแบบแอป
ResizeObserver	ปรับขนาดพริ้วกล้องให้พอดีจอทุกแนว

4 ฟีเจอร์และเทคนิคที่ใช้

 ฉากเวทมนตร์ (ตัดคน) - MediaPipe Image Segmenter, delegate = CPU (ทำงานเหมือนกันทุกเบราว์เซอร์) - ตัดคนที่ละข้อระหว่างพัก จีรอลหลังข้อสุดท้ายยาว 3 วินาที - ปรับขอบคนให้นุ่ม, ปรับโทนสีคน, เรืองแสงรอบตัว (light wrap) - ใช้กับ JPG เท่านั้น มีปุ่มสลับกลับเป็นฉากจริง	 ฟิลเตอร์สี - ปกติ · ทองอ่อน · คำคีนน้ำเงิน · ขาวดำ · เทามูดี · ฟิล์มซีเปีย - คำนวณเองที่ละพิกเซล (ไม่ใช่ <code>ctx.filter</code> เพราะ Safari บางรุ่นไม่รองรับ) - ลงเฉพาะรูปถ่ายใน JPG และ GIF ไม่ลงกรอบ
 กรอบรูป - เทมเพลตพิมพ์ 1024x1536 อ่านช่องรูปจาก <code>templates.json</code> - กรอบเดิม 1200x1800 ตั้งค่าใน <code>frames.ts</code> มีกรอบสำรองที่วาดด้วยโค้ด - วาดตามลำดับ: พื้นหลัง → รูป (ครอป cover) → overlay	 เสียง / ✨ ประกาย - เสียงสร้างด้วย Web Audio จึงไม่มีปัญหาลิขสิทธิ์ และมีปุ่มปิดเสียง - ประกายเป็นอนุภาคบน Canvas ด้วย <code>requestAnimationFrame</code> - รองรับ <code>prefers-reduced-motion</code> (ลดการเคลื่อนไหว)

5 ไฟล์ผลลัพธ์

ไฟล์	ขนาด / รูปแบบ	รายละเอียด
JPG รูปรวม	1024x1536 (เทมเพลต) / 1200x1800 (กรอบเดิม)	quality 0.92 มีกรอบ ฉากเวทมนตร์ และฟิลเตอร์
GIF	600 px กว้าง (~732 px สูง)	รูป 4 ข้อไม่มีกรอบ ฉากจริง มีฟิลเตอร์ เล่นวนไม่สิ้นสุด

พิมพ์	4x6 นิ้ว แนวตั้ง	ผ่าน @page ความละเอียดประมาณ 256 dpi
ชื่อไฟล์	magical-booth-YYYYMMDD-HHmss	ตามเวลาเครื่อง

6 UI / การออกแบบ

CSS	เขียนเองทั้งหมด ไม่ใช้ framework ใช้ CSS variables, รองรับแนวตั้ง/แนวนอน, dvh และ safe-area สำหรับ iPad
ฟอนต์	Mitr (ภาษาไทย) และ Cinzel Decorative (หัวเรื่องอังกฤษ) จาก Google Fonts ถ้าไม่มีเน็ตจะใช้ฟอนต์ของเครื่องแทน
Kiosk	กัน zoom / scroll / เลือกข้อความ / เมนูกดค้าง, กลับหน้าแรกเองเมื่อไม่มีคนใช้ 60 วินาที

7 แพลตฟอร์มเป้าหมายและการทดสอบ

เบราว์เซอร์	Chrome, Edge, Safari (macOS) และ Safari บน iPad ทั้งแนวตั้งและแนวนอน · build target: ES2020 / Safari 14 / Chrome 90 / Edge 90
ทดสอบอัตโนมัติ	Playwright + Chromium กับกล้องจำลองในเครื่องพัฒนา (สคริปต์ไม่ได้อยู่ใน repo)
ยังไม่ได้ทดสอบ	Safari และ iPad จริง, ความเร็วการตัดคนบน iPad, เสียง, การพิมพ์กับเครื่องพิมพ์จริง

8 แผนถัดไป (ยังไม่ได้ทำ)

เฟส	ฟีเจอร์	เทคโนโลยีที่คาดว่าจะใช้
B	สติ๊กเกอร์ลากวาง, คลิปเบื้องหลัง + boomerang	Pointer Events, MediaRecorder (MP4/WebM)
C	QR Code ให้แขกโหลดรูป/GIF/คลิป, หน้าตั้งค่าลับ + โปรไฟล์ event	Cloudflare R2 + Pages หรือ Firebase Storage + Hosting, ไลบรารีสร้าง QR

หมายเหตุ: ตอนนี้ไม่มี server หรือบริการคลาวด์ ค่าใช้จ่ายจึงเป็น 0 บาท เฟส C จะเริ่มต้องใช้บัญชีคลาวด์และอินเทอร์เน็ตที่บูธ